

Spec — Operations Monitoring: Mac + EC2 Division of Labor

Created: 2026-06-22 — Claude.ai Status: Design — build this week after Block B German batch Priority: Medium — system is running, but nothing is watching EC2

Context

mini-moi now runs across two nodes. Mac has monitoring that evolved organically (disk alerts, health checks via OpenClaw). EC2 has nothing watching it. With EC2 as production, this is a real gap.

Additionally the monitoring responsibilities between OpenClaw, CoS, and the system bot are undefined. This spec clarifies the division of labor and what gets built.

EC2 is production and deserves production-grade operations. CoS role is escalation and informing — detect problems, alert Robert, provide context for his decision. CoS does not self-heal or restart services. Robert decides what action to take. This keeps the human in control of production changes.

Division of Labor (locked)

Agent	Scope	Nature
OpenClaw	Independent system-wide checks — spans beyond minimoi	Proactive, autonomous, no hard dependency on minimoi
CoS	minimoi-specific operational health	Interactive, responds to queries, owns EC2 monitoring task
minimoi_system_bot	Delivery channel for alerts	Sends notifications, not the checker

Overlap is intentional and fine. OpenClaw and CoS both checking disk, memory, or container health is redundant in a good way — neither is a single point of failure.

Current state

Mac (working but informal):

- OpenClaw runs independent checks (disk, process health)

- `minimoi_system_bot` sends disk alerts proactively
- `minimoi_cos_bot` handles interactive health queries
- Comment in code still references `rvsopenbot` — stale, needs cleanup

EC2 (nothing watching it):

- No health checks running
 - No alerts if containers go down
 - No disk/memory monitoring
 - CoS has no visibility into EC2 operational state
-

What to build

Part 1 — EC2 health monitoring (CoS task)

Add a periodic CoS task that checks EC2 operational health and escalates to Robert via `minimoi_system_bot` if something needs attention. CoS detects and informs — Robert decides what to do.

Two-layer tooling approach:

Layer 1 — AWS-native (CloudWatch): CloudWatch already collects EC2 instance metrics. Use it for:

- EBS disk utilization (already instrumented by AWS)
- Instance-level CPU and memory
- Network in/out These are AWS-specific but already available at no extra cost. CloudWatch alarms can trigger SNS → `system_bot` notifications for threshold breaches without any application code.

Layer 2 — Standard Linux tools (portable): These checks run inside the container or via subprocess and would work on any future machine (Mac Mini, another cloud):

- `docker ps` — are all containers running?
- `df -h` — disk usage on the mounted volume
- `free -m` — memory pressure
- `curl localhost:5001/health` — Flask app responding?

Using both layers means AWS metrics for infrastructure visibility and portable Linux checks for application health. Neither alone is sufficient.

Checks to run every 30 minutes:

- All four containers running: portal, curator, german, postgres
- Disk usage < 80% (both CloudWatch EBS metric + df -h)
- Memory < 85%
- `/health` endpoint on all three Flask apps returns 200

Alert conditions — CoS escalates, does not fix:

- Any container not running → immediate alert: "Container X is down on EC2. Needs attention."
- Disk > 80% → alert with current usage and largest directories
- Memory > 85% → alert with top memory consumers
- Any `/health` non-200 → alert with which app and status code

CoS alert format (informative, actionable):

```

! EC2 Health Alert
Container: minimoi-curator is not running
Last seen: 14 minutes ago
Action needed: Check docker logs minimoi-curator on EC2
To restart: docker-compose up -d curator

```

Robert sees the problem, context, and the exact command to fix it. CoS does not run that command.

Implementation:

```

python

# In chief_of_staff.py - new periodic task
def check_ec2_health():
    """
    Runs on EC2 only (is_production() guard).
    Detects problems and escalates to Robert via system_bot.
    Does not attempt to fix anything.
    """
    if not is_production():
        return

    # Layer 2: standard Linux checks
    # Layer 1: query CloudWatch metrics via boto3
    # Alert via get_system_token() if thresholds breached

```

CoS checks run inside the EC2 container — no SSH needed, uses subprocess for Linux tools and boto3 for CloudWatch. HTTP for Flask health endpoints.

Part 2 — Mac monitoring cleanup

Clean up the stale `(rvsopenbot)` reference in the Mac monitoring code. Confirm all Mac alerts route through `(minimoi_system_bot)` via `(get_system_token())`.

Also: confirm `(minimoi_cmd_bot)` and `(com.user.telegram-feedback-bot)` launchd jobs are unloaded and disabled on Mac (these were marked for retirement after system_bot took over).

Part 3 — OPERATIONS.md update

Update `(docs/OPERATIONS.md)` with current two-node state:

Sections to add/update:

- Two-node architecture (EC2 production, Mac standby)
 - What CoS monitors on EC2 (container health, disk, memory)
 - What OpenClaw monitors independently (broader system checks)
 - Alert routing (all alerts → minimoi_system_bot)
 - How to check EC2 health manually (docker ps, /health endpoints)
 - Colima maintenance note for Mac (from OpenClaw handoff)
 - How to restart containers on EC2 if something goes down
-

SSM / credential note

EC2 health checks need no new credentials — they run inside the EC2 environment with existing IAM role. No new SSM parameters needed.

What OpenClaw handles (out of scope for this spec)

OpenClaw keeps its own independent checks that span beyond minimoi. These are not part of this spec. OpenClaw to maintain and evolve independently. No hard dependency between OpenClaw checks and this CoS monitoring task.

Definition of Done

- ☐ CoS periodic health check running on EC2 (30 min interval)

- ☐ Alerts fire via minimoi_system_bot when containers down, disk > 80%, memory > 85%, or /health non-200
- ☐ Stale rvsopenbot reference removed from Mac monitoring code
- ☐ All Mac alerts confirmed routing through get_system_token()
- ☐ minimoi_cmd_bot and telegram-feedback-bot launchd jobs unloaded and disabled on Mac
- ☐ docs/OPERATIONS.md updated with two-node architecture, monitoring responsibilities, and manual check procedures
- ☐ Verified: kill a container on EC2, confirm alert arrives within 30 minutes via system_bot

Commit sequence

1. Add check_ec2_health() to chief_of_staff.py with is_production() guard
2. Mac monitoring cleanup – remove rvsopenbot reference
3. Unload retired Mac launchd jobs
4. Update docs/OPERATIONS.md

Commit message

Operations: EC2 health monitoring via CoS, Mac cleanup, OPERATIONS.md updated for two-node architecture.

Spec · 2026-06-22 · Claude.ai Build after: Block B German batch Register in design_log as: spec_ready